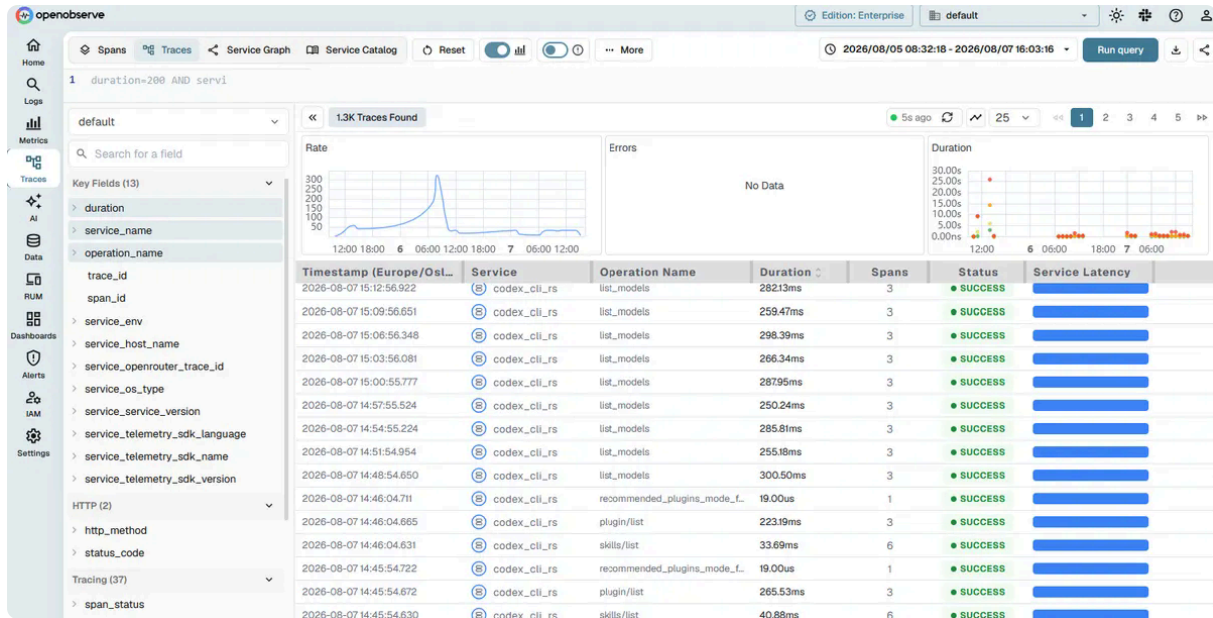


Measuring what a coding agent actually costs

Emil Lindfors | August 13, 2026



The two previous posts in this series were about pointing a coding agent at **whichever endpoint a client will approve**, and **the code that made that possible**. Both of them make cost claims. This one is about how I know those numbers, which took considerably longer than the fork did.

Everything here is self-hosted and open source. No vendor SaaS, no per-seat pricing, and no shipping prompt content to a third party — which, as it turns out, is a thing you have to actively prevent.

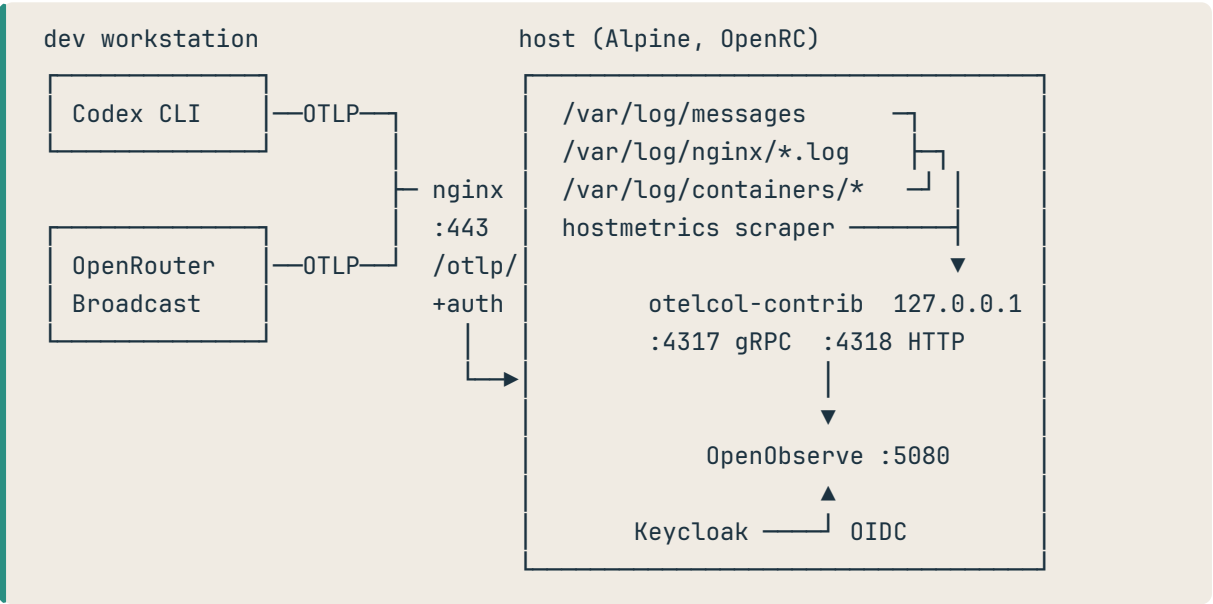
The stack

Component	Role	Version
OpenObserve	Store, query, dashboards for logs, metrics and traces	latest, podman container
OpenTelemetry Collector (otelcol-contrib)	Ingest, transform, route	0.158.0

Component	Role	Version
Keycloak	OIDC SSO in front of OpenObserve	realm <code>lindfors</code>
nginx	TLS and an authenticated public OTLP endpoint	—
Codex CLI	LLM client, emits its own OTLP metrics and traces	0.1.1
OpenRouter Broadcast	LLM gateway, ships spans with token counts and cost	—

OpenObserve Enterprise is free under 200 GB/day of ingest, which is not a constraint for one host.

Three ingest paths, one collector, one store:



OpenObserve speaks OTLP directly, so strictly the collector is optional. It earns its place by routing to per-destination streams (OpenObserve picks the stream from a `stream-name` header, so you need one exporter per stream, otherwise everything lands in `default` and retention becomes unmanageable), and by dropping noise before it costs storage. That second job turned out to matter more than anything else here.

The OTLP receiver binds to loopback and has no auth of its own. Anything from off-host arrives through nginx, which terminates TLS, enforces basic auth, and strips the credential before proxying so the collector never sees it:

```
location /otlp/ {
    auth_basic "OTLP ingest";
    auth_basic_user_file /etc/nginx/otlp.htpasswd;

    proxy_pass http://127.0.0.1:4318/;
```

```
# do not leak the ingest credential to the collector
proxy_set_header Authorization "";

client_max_body_size 32m;
}
```

`client_max_body_size 32m` is not decoration. OTLP batches carrying full prompt text get large and the 1 MB default silently 413s them.

Getting the numbers, three times

This is the part worth writing about, because I got it wrong twice.

Attempt 1: derive metrics from OpenRouter spans

OpenRouter Broadcast emits a span per generation with token counts and cost as attributes. You can query the trace stream directly, but that binds cost history to trace retention and gets slow over long ranges. So: convert spans into real metric time series with the collector's `sum` connector.

```
connectors:
  sum:
    spans:
      gen_ai.usage.input_tokens:
        source_attribute: gen_ai.usage.input_tokens
        conditions:
          - 'attributes["gen_ai.usage.input_tokens"] != nil'
        attributes:
          - key: gen_ai.request.model
```

Two traps in that block, both of which I only found by attaching the debug exporter and reading raw datapoints.

The emitted value is multiplied by the number of `attributes` entries. Listing two keys turned a 500-token span into 1000. Keep exactly one.

A span missing any listed attribute is dropped silently. No datapoint, no warning, no metric. Listing `gen_ai.system` alongside the model would have discarded every span that lacked it, and I'd have had a chart that looked fine.

A third one came from the `hostmetrics` side. The `sum` connector emits Delta temporality, so `sum()` over a time bucket is correct as written. Other sources emit **cumulative** counters, where each datapoint is a running total, and summing those per hour double-counts every snapshot. An hour of about 1.08 M input tokens came out as roughly 1.6 M.

```
processors:
  cumulative_to_delta:
    include:
      metrics: [gen_ai.usage.input_tokens, gen_ai.usage.output_tokens,
gen_ai.usage.total_cost]
      match_type: strict
```

Find out which temporality your source emits before writing a single aggregation. None of these three produced an error. They produced numbers that were plausibly wrong, which is the worst kind.

Attempt 2: use Codex's own metrics

Codex CLI exports its own OTLP metrics, and they look like the right source: first-party, no span-derivation fragility, about 35 metric families covering turns, tool calls, latency, SSE events, sqlite init, skills selection.

The shape differs from the OTEL `gen_ai` semantic conventions, so a few notes if you go this way. All token counts live in one histogram, `codex_turn_token_usage`, split by a `token_type` label (`input`, `cached_input`, `cache_write_input`, `output`, `reasoning_output`, `total`) rather than one metric per direction; the `_sum` stream carries the count and `_count` is the number of turns. `token_type = 'total'` already includes input plus output, so any per-model panel has to filter to a single type or it double-counts. And the model label is plain `model`, not `gen_ai.request.model`.

Two things killed it as the primary source.

Codex reports no cost metric at all. There is no `codex_*` equivalent of `gen_ai.usage.total_cost`. Token counts are there, so you could reconstruct cost from your own price table, but nothing on the wire carries it.

And the token metrics report zero. `codex_turn_token_usage` records `0` for every `token_type`, including `input` and `total`, across heavy multi-hour sessions — all 16 histogram buckets land in `le=0`. The same zeros show up in Codex's own `session_task.turn` spans, which rules out the collector and the store. It's Codex's accounting.

Meanwhile the latency and activity metrics from the same source are fine and report real numbers. A telemetry source can be half-broken, and on a dashboard the broken half looks exactly like “no usage yet”.

Attempt 3: each source for what it's good at

What stuck:

Signal	Source	Why
Cost (total, input, output, unit prices)	OpenRouter <code>LLM Generation</code> spans	Only source that has it
Tokens (input, output, cached)	OpenRouter <code>LLM Generation</code> spans	Codex's are zero
Requests, turns	Codex metrics	Client-side truth, including failures
Latency (turn end-to-end, TTFT)	Codex metrics	Measured where the user waits

The trap on the OpenRouter side is that a single generation produces a span tree, and only the root carries real numbers:

```

LLM Generation          inp=92495  out=535  total_cost=0.001859  ←
the real one
├ generation            inp=0       out=0       cost=-0.0
└ provider attempt 1: DeepInfra inp=0       out=0       cost=-0.0

```

Filter on `operation_name = 'LLM Generation'` or every number is zero. I nearly built the whole dashboard on a child span.

Cost fields also arrive as strings. `gen_ai_usage_total_cost` is `"0.001859058"`, not a float, so every panel needs `CAST(... AS DOUBLE)`. Token counts are numeric. There's no consistency to it, it's just a thing to know.

Trying to join the two sources

The obvious next question is whether you can attach a cost figure to a specific Codex turn, or to a project. As far as I can tell, not without changing something upstream.

What I checked:

- **Trace context.** Zero shared `trace_id`s. Either Codex doesn't inject `traceparent` into the OpenRouter call, or OpenRouter doesn't honour it.
- **A client-supplied id.** OpenRouter surfaces `trace_metadata.openrouter.user_id` from the `user` field of the request body. Codex sends nothing there.
- **Wall clock.** The only real link. Codex's `chat.stream_request` trails OpenRouter's `LLM Generation` by a strikingly stable 227–242 ms, with 5–48 s between calls, so pairing them by eye is unambiguous.

That last one is a trap dressed up as a solution, and I abandoned it for two reasons.

The first is that OpenObserve's DataFusion silently returns zero rows for inequality joins. An interval join (`ON o.start_time BETWEEN c.start_time AND c.end_time`) returns an empty result set. Not an error, not a warning, just nothing. Equi-joins are fine. `UNION ALL` and cross joins panic outright:

```
assertion failed: variadic_counts.is_empty()
```

So the only workable shape is an equi-join on a bucketed timestamp, which means hard-coding a 235 ms offset and hoping it never drifts across a collector restart, a network change or an NTP correction. That is not something I want behind a number I show a client.

What works instead, for free: a separate OpenRouter API key per project. The key name arrives as `trace_metadata.openrouter.api_key_name` on every span, which gives exact project-level attribution with no join at all. It's coarser than per-turn and it's correct, and I'll take correct over precise.

If you want to correlate two telemetry sources, the join key has to be designed in from the start.

Agent traces are enormous

Codex with tracing on emitted **118,469 spans in one working session**. 118 of them were about the LLM. The rest is Rust async runtime internals:

```
FramedRead::poll_next      1626
poll_ready                 1626
try_reclaim_frame          1527
FramedRead::decode_frame   864
FramedWrite::flush         838
pop_frame                  825
...
```

0.1% signal. h2 frame handling and tokio worker polls, faithfully shipped across the network and stored forever. This is the biggest operational problem in the whole setup, bigger than any of the metric bugs, because it's pure cost with no upside at all.

The store does the same thing to itself. OpenObserve at `INFO` logs every ingest and every search. The collector tails container logs and ships them to OpenObserve, which logs that, which gets shipped back. About 10k lines an hour of pure self-reference. Two settings are needed to stop it, because `ZO_LOG_LEVEL` on its own does nothing — the Rust `tracing` subscriber honours `RUST_LOG`, and that's the one that takes effect:

```
ZO_LOG_LEVEL=warn
RUST_LOG=warn
```

There's a third variant neither setting touches. OpenObserve dumps DataFusion physical query plans to stdout during compaction, printed as raw table output rather than through the tracing subscriber, so no config flag suppresses them. Those get dropped at the collector:

```
processors:
  filter/plan_dumps:
    error_mode: ignore
  logs:
    log_record:
      - 'IsMatch(body, "(DataSourceExec|ProjectionExec|AggregateExec|SortExec|FilterExec)")'
      - 'IsMatch(body, "file_groups=\\{")'
      - 'IsMatch(body, "merge_parquet_files")'
```

Budget for noise suppression as a real part of the work. Every component here over-reports by default and each one needed a different mechanism to quiet down: an env var, a second env var, a collector filter, and a client-side sampling policy I still haven't written.

Your prompts end up in your logs

By default OpenRouter Broadcast ships `gen_ai.input_messages` and `gen_ai.output_messages`, which is the entire conversation including the system prompt. On this setup that means 90k-token prompts sitting in the trace store. It's a privacy question and it's also most of the storage.

Turning on LLM tracing means logging your prompts unless you take a specific step to stop it. There's a provider-side toggle; the collector-side attribute drop is the belt-and-braces version and it covers every source at once:

```
processors:
  transform/strip_prompt_content:
    error_mode: ignore
  trace_statements:
    - context: span
      statements:
        - delete_key(attributes, "gen_ai.input_messages")
        - delete_key(attributes, "gen_ai.output_messages")
        - delete_key(attributes, "gen_ai.system_instructions")
```

Which I have written down and not yet applied on this host, so take it as a recommendation from someone who hasn't finished following it.

The dashboard

One dashboard, `LLM Usage`, ten panels, both sources on a shared hourly axis and read together by eye rather than joined.

Panel	Source
Cost (USD) per hour by model	OpenRouter traces
Total cost by model	OpenRouter traces
Cost by upstream provider	OpenRouter traces
Total tokens by model	OpenRouter traces
Tokens per hour by model	OpenRouter traces
Cached input tokens per hour	OpenRouter traces
API requests per hour by model	Codex metrics
Conversation turns by model	Codex metrics
Slowest turn end-to-end (ms)	Codex metrics
Slowest time to first token (ms)	Codex metrics

A representative panel, with both the `CAST` and the root-span filter:

```
SELECT histogram(_timestamp, '1 hour')           as "x_axis_1",
       sum(CAST(gen_ai_usage_total_cost AS DOUBLE)) as "y_axis_1",
       gen_ai_request_model                       as "breakdown_1"
FROM   "default"
WHERE  operation_name = 'LLM Generation'
GROUP BY "x_axis_1", "breakdown_1"
ORDER BY "x_axis_1"
```

Dashboards are managed over the REST API rather than the UI, which makes them reviewable and re-creatable:

```
GET  /api/{org}/dashboards
PUT  /api/{org}/dashboards/{id}?folder=default&hash={hash}
```

The `hash` is a concurrency token from the `GET`. Stale hash, rejected write.

Sample numbers over about two days of real use: **\$0.184 across 5.97 M tokens, of which 2.0 M were prompt-cache hits**. The cache panel is the most actionable thing on the board, since it's the difference between an eighteen-cent day and a much worse one. That figure is also the evidence behind the cost claims in the [business post](#) — at this scale tooling cost is noise, but I'd rather say so with a number than assert it.

Access control, briefly

OpenObserve's Dex integration points straight at Keycloak's OIDC endpoints, with no Dex container involved. Four things that cost me time:

`O2_DEX_BASE_URL` must be the realm root, because OpenObserve fetches `BASE_URL + /.well-known/openid-configuration` and Keycloak serves that at the realm root rather than under `/protocol/openid-connect`. The Keycloak client needs an audience mapper adding

`openobserve` to the access token's `aud`, since Keycloak defaults it to `account`, which fails validation. Org and role come from per-user Keycloak attributes (`o2_org` / `o2_role`) surfaced as claims by attribute mappers, deliberately not the `groups` claim, whose value is `admin` and would put users in a non-existent `admin` org while all data ingests into `default`; those attributes also have to be declared in the realm's user profile or Keycloak drops them without saying so. And roles of external users can't be edited in OpenObserve at all, because they're read from these claims on every login.

Container logs get their service name from the filename, which means starting containers with an explicit log path:

```
podman run --log-driver k8s-file --log-opt path=/var/log/containers/<name>.log
```

The default podman path exposes only a container ID. An earlier version of my config hardcoded `source: keycloak` onto every container's logs, which was simply wrong — Keycloak's log is about 4 KB where OpenObserve's is 1.4 MB over the same window, so the mistake was easy to spot once I looked.

Still open

- **Codex trace sampling.** 99.9% of spans are runtime internals. Needs a client-side filter, or a `filter` processor on `service.name IN (codex_*)` that drops everything but the handful of meaningful operations.
- **Stripping prompt content** at the collector, per above.
- **Retention.** `/opt/openobserve/data` is 238 MB and growing, and retention isn't configured, so it grows without bound.
- **Codex token metrics reporting zero**, which I should report upstream rather than just work around.
- The `sum` connector and its `cumulative_to_delta` entry are dead weight now that cost is read from spans directly, and I haven't removed them.

If you build this yourself

Attach the debug exporter before anything else. Every metric bug above was invisible in the dashboard and obvious in raw datapoints.

Check temporality before writing any aggregation, since cumulative summed per bucket is the easiest way to be 50% wrong. Find the root span, because span trees put the real numbers in one place and zeros everywhere else. Design the join key in or don't join. Verify a query returns rows before trusting an empty result, because a silent zero-row join looks exactly like "no data yet". Assume every component over-reports and that each

needs a different off switch. And decide about prompt content on day one, not after 90k-token prompts are already on disk.

If you've found a way to get a real join key between an agent and a gateway without patching one of them, I'd like to hear it, because the per-key attribution I settled on is coarser than I want.